



***Universidad Tecnológica
de El Salvador***

Facultad Informática y Ciencias Aplicadas

Estándares de Programación

Sección: 01

Tema:

Cuarta y Quinta entrega avance de proyecto.

Nombre del docente:

Ing. Ruben Escobar Ortegon.

Nombre del estudiante:

José Rigoberto Guerra Rodríguez-----	25-1083-2022
Christian Osvaldo Henríquez Escalante-----	25-0818-2022
Bryan Eduardo Jiménez Mendoza-----	25-2055-2022
Axel Antonio Martínez Portillo-----	25-2318-2022
Elmer Josué Menéndez Reinosá-----	25-1660-2022
Oscar Iván Moreno López -----	25-0029-2020

Fecha: 22/05/2026

Transforma tu realidad.
Viví la experiencia **UTEC**



Universidad Tecnológica de El Salvador

Objetivos

- General:

Desarrollar una plataforma web para digitalizar la gestión, ejecución y seguimiento de auditorías internas de la empresa textil Robles S.A de C.V.

- Específicos:

- Analizar los procesos de auditoría interna de una empresa textil para identificar los requerimientos del sistema.
- Modelar la base de datos necesaria para almacenar auditorías, hallazgos.
- Implementar la generación automática de reportes de auditoría.



Universidad Tecnológica de El Salvador

Entrega Final (Unidad 4): Plan de Pruebas y Calidad

Plan de Pruebas

1. Alcance de las Pruebas

Dentro del Alcance: Nos enfocaremos en la validación de la lógica de negocio y la comunicación entre tus tres capas (Frontend en React, Backend en Node.js y Base de Datos en PostgreSQL).

- Módulo de Autenticación: Validación del inicio de sesión y la correcta emisión del token.
- Control de Accesos (Roles): Verificación estricta de que las reglas de negocio se cumplan: un Administrador tiene control total, y un Auditor está restringido a sus tareas operativas.
- Gestión de Auditorías (CRUD): Creación, lectura, actualización y eliminación de reportes de auditoría.
- Integración de la API: Confirmar que los datos JSON viajen correctamente desde la interfaz hasta la base de datos sin pérdida de información.

Fuera del Alcance:

- Pruebas de Carga o Estrés: No mediremos cuántos miles de usuarios soporta el servidor simultáneamente.
- Pruebas de Compatibilidad Móvil: Solo se probará el comportamiento responsivo en navegadores web de escritorio, no en dispositivos móviles nativos.



Universidad Tecnológica de El Salvador

2. Tipos de Prueba y Herramientas a Utilizar

A cada riesgo del sistema le asignamos un tipo de prueba y una herramienta específica para mitigarlo.

- Pruebas de Integración de API (Backend)
 - Objetivo: Asegurar que las rutas de Node.js se comuniquen correctamente con PostgreSQL y devuelvan las respuestas JSON esperadas.
 - Herramienta: Thunder Client. Se crearán "Collections" para ejecutar peticiones GET, POST, PUT y DELETE automatizadas contra los *endpoints* locales.
- Pruebas de Seguridad y Autorización (RBAC)
 - Objetivo: Intentar "hackear" los permisos. Probaremos inyectar tokens modificados o consumir rutas de Administrador con el token de un Auditor.
 - Herramientas: Thunder Client (para manipulación de cabeceras *Bearer Auth*) y jwt.io (para decodificar y revisar la estructura del token generado).
- Pruebas Funcionales y de Interfaz (Frontend)
 - Objetivo: Validar que los componentes de React reaccionen correctamente. Por ejemplo, que los botones de "Eliminar Auditoría" no existan en la pantalla si el usuario es un Auditor.



Universidad Tecnológica de El Salvador

- Herramientas: Navegador web (Chrome/Edge) utilizando Chrome DevTools (pestañas *Network* para ver el tráfico de red y *Application* para verificar el almacenamiento del JWT en LocalStorage/Cookies).
- Pruebas de Caja Negra
 - Objetivo: Comportamiento ante el error humano. Ingresar contraseñas vacías, correos sin formato válido, o letras en campos numéricos para confirmar que el sistema lanza alertas y no se cae.
 - Herramientas: Formularios de la interfaz web en React.

3. Ambiente de Pruebas

Para el SAR, el ambiente estará configurado de la siguiente manera:

- Arquitectura: Entorno de desarrollo local (*Localhost*).
- Base de Datos (PostgreSQL): Una base de datos dedicada exclusivamente a pruebas
- Servidores:
 - Backend (Node.js) ejecutándose en http://127.0.0.1:3000 gestionado con npm.
 - Frontend (React) ejecutándose en el puerto de desarrollo (ej. http://localhost:5173).
- Datos Semilla: La tabla sar_usuarios debe estar pre-cargada con al menos dos usuarios de control con los que trabajará el equipo de QA:
 - Usuario 1: admin_prueba (Rol: Administrador).



Universidad Tecnológica de El Salvador

- Usuario 2: auditor_prueba (Rol: Auditor).

4. Criterios de Aceptación

Son las reglas de oro; si el sistema no cumple alguna de estas condiciones, la prueba se marca como fallida y se devuelve a desarrollo.

Para el Módulo de Autenticación:

- Criterio 1: Si un usuario ingresa credenciales válidas registradas en PostgreSQL, el servidor DEBE responder con un código HTTP 200 OK y devolver un Token JWT que contenga el username y su rol.
- Criterio 2: Si las credenciales son incorrectas, el servidor DEBE responder con un HTTP 401 Unauthorized y no debe generar ningún token.

Para el Control de Accesos (Roles):

- Criterio 3: Si un usuario autenticado con rol de "Auditor" intenta realizar una petición POST a una ruta administrativa el Node.js DEBE bloquear la acción y retornar un código 403 Forbidden.
- Criterio 4: La interfaz de React DEBE leer el JWT y ocultar visualmente los menús de configuración administrativa a cualquier usuario que no tenga el rol explícito de "Administrador".

Para la Gestión de Auditorías:

- Criterio 5: Al registrar una nueva auditoría desde el frontend, los datos DEBEN guardarse en PostgreSQL vinculando de forma automática e invisible el ID o username del auditor que realizó la petición.



Universidad Tecnológica de El Salvador

- Criterio 6: Al expirar el tiempo de vida del Token JWT, cualquier petición posterior hacia el backend DEBE ser rechazada y React DEBE redirigir al usuario automáticamente a la pantalla de Login.

Matriz de Pruebas

ID	CASO DE PRUEBA	PASOS DE EJECUCION	RESULTADO ESPERADO	EVIDENCIA
TC-01	Login Exitoso (Admin)	En Thunder Client, enviar POST a /api/login con credenciales válidas de Administrador.	Código HTTP 200. El servidor devuelve mensaje de éxito y el token JWT generado.	Ejecutado: Captura de pantalla de Thunder Client mostrando el JSON con el token.
TC-02	Login Fallido (Credenciales Inválidas)	Enviar POST a /api/login con una contraseña errónea o usuario inexistente.	Código HTTP 401. El servidor devuelve JSON con error "Usuario o contraseña incorrectos".	Ejecutado: Captura de pantalla de Thunder Client mostrando el error 401.
TC-03	Interfaz React por Rol (Auditor)	Iniciar sesión en React con credenciales de Auditor. Revisar el menú lateral.	Opciones administrativas (ej. "Crear Usuario") están invisibles en la interfaz de React.	Ejecutado: Captura de pantalla del menú lateral en el navegador (solo opciones de auditoría).



Universidad Tecnológica de El Salvador

TC-04	Bloqueo de API por Rol (RBAC)	En Thunder Client, enviar POST a ruta de admin (/api/usuarios) usando un token de Auditor en Auth Bearer.	Código HTTP 403 (Forbidden). El backend rechaza la acción por permisos insuficientes.	Ejecutado: Captura de pantalla de Thunder Client mostrando el rechazo 403.
TC-05	Registrar nueva Auditoría	En React: abrir formulario de Nueva Auditoría, llenar campos y guardar.	Se guarda en PostgreSQL y la nueva auditoría aparece en el listado principal de React.	Ejecutado: Captura de pantalla dual (Interfaz con el registro guardado y tabla en pgAdmin).
TC-06	Validación de Formulario Vacío	En React: hacer clic en "Guardar" en el formulario de Auditoría sin llenar datos obligatorios.	El frontend detiene la petición. Aparecen alertas rojas exigiendo los campos obligatorios.	Ejecutado: Captura de pantalla del formulario en React mostrando los mensajes de validación.
TC-07	Listado General de Auditorías	Navegar a la pantalla principal de Auditorías en React.	La tabla se llena con todos los registros extraídos desde PostgreSQL (HTTP 200).	Pendiente



Universidad Tecnológica de El Salvador

TC-08	Actualizar Auditoría	En React: seleccionar una auditoría existente, modificar una nota y guardar.	Los datos se actualizan en la base de datos y la vista se refresca mostrando el cambio.	Pendiente
TC-09	Eliminar Auditoría (Admin)	En React: entrar como Admin, clic en "Eliminar" sobre una auditoría y confirmar.	El registro desaparece de la vista y se elimina permanentemente de PostgreSQL.	Pendiente
TC-10	Eliminar Denegado (Auditor)	En Thunder Client, intentar enviar DELETE a /api/auditorias/1 usando token de Auditor.	Código HTTP 403. El sistema protege el registro para evitar borrados no autorizados.	Pendiente
TC-11	Crear Nuevo Usuario (Admin)	En React: Administrador llena formulario de nuevo usuario y asigna rol de "Auditor".	El usuario se inserta en sar_usuarios y ya puede iniciar sesión en el sistema.	Pendiente
TC-12	Manejo de Sesión Expirada	Usar un token JWT vencido (o alterado) para solicitar el listado de auditorías.	Código HTTP 401. React lo detecta, destruye la sesión y redirige a la pantalla de Login.	Pendiente



Universidad Tecnológica de El Salvador

Registro de Defectos (Bug Log)

ID DEFECTO	SEVERIDAD	DESCRIPCION CORTA
BUG-01	ALTA	Al consultar la ruta /api/usuarios con un token de Auditor, el sistema devuelve la lista completa en lugar de bloquear el acceso.
BUG-02	CRITICA	Al enviar una petición sin el encabezado Authorization, el servidor de Node.js colapsa (Crash) y detiene el servicio.
BUG-03	MEDIA	Si el JWT expira, al presionar "Guardar auditoría" la pantalla de React se congela en lugar de redirigir al login.
BUG-04	ALTA	Al registrar una auditoría con comillas simples en las observaciones, PostgreSQL devuelve un Error 500.
BUG-05	BAJA	El formulario de creación de usuarios en React permite registrar cuentas con contraseñas en blanco.

Checklist de usabilidad

ID	ITEM EVALUADO	DESCRIPCION DEL CRITERIO	ESTADO ACTUAL	OBSERVACIONES
UX-01	Validación de campos obligatorios	El sistema indica visualmente (asteriscos, bordes rojos) qué datos son requeridos antes de permitir guardar una auditoría.	PARCIAL	Falla en el formulario de "Crear Usuario" (Relacionado al BUG-05). En auditorías funciona bien.
UX-02	Mensajes de error claros	En lugar de mostrar errores técnicos de PostgreSQL (ej. códigos SQL), el	NO CUMPLE	El sistema muestra alertas técnicas o simplemente se congela.



Universidad Tecnológica de El Salvador

		sistema traduce el problema a lenguaje humano.		
UX-03	Feedback visual (Estados de carga)	Al hacer clic en "Guardar" o "Iniciar Sesión", un <i>spinner</i> de carga indica que el sistema está trabajando, deshabilitando el botón para evitar doble clic.	CUMPLE	Implementado correctamente. Previene la inserción de registros duplicados en la base de datos.
UX-04	Confirmación en acciones destructivas	Si el Administrador presiona "Eliminar Auditoría", aparece un cuadro de diálogo tipo <i>modal</i> preguntando "¿Está seguro de eliminar este registro?".	CUMPLE	Previene el borrado accidental de información crítica del SAR.
UX-05	Prevención de pérdida de datos	Si el usuario está llenando un reporte de auditoría y presiona "Atrás" o cierra la pestaña por error, el sistema le advierte que tiene cambios sin guardar.	NO CUMPLE	Actualmente el usuario pierde todo el texto ingresado si navega hacia otra pantalla.
UX-06	Navegación adaptada al Rol (Menos ruido)	La interfaz oculta completamente las funciones administrativas para los Auditores, dejando una pantalla limpia y enfocada	CUMPLE	Reduce la carga cognitiva y mejora el enfoque de los empleados.



Universidad Tecnológica de El Salvador

		solo en sus tareas operativas.		
UX-07	Legibilidad en grandes volúmenes de datos	La tabla principal de auditorías cuenta con paginación (ej. muestra de 10 en 10) y buen contraste de colores, evitando que la pantalla se sature.	CUMPLE	Facilita la lectura y mejora el rendimiento visual del navegador.
UX-08	Accesibilidad básica (Navegación por teclado)	Es posible moverse entre los campos del formulario de inicio de sesión y el de Nueva Auditoría presionando únicamente la tecla Tab.	CUMPLE	Ideal para agilizar el ingreso de datos en entornos de oficina.



Universidad Tecnológica de El Salvador

Entrega Final (Unidad 5): Prototipo + Gestión

Resultado del sistema y evidencias Capturas de los flujos

Promesa	Resultado Final	Estado Actual
Autenticación segura de usuarios	Se implementó un sistema de inicio de sesión utilizando username y encriptación con JWT de contraseñas, validando roles.	Completado.
Gestión de Plantillas de Auditoría	El sistema permite registrar plantillas y agregarles preguntas dinámicas.	Completado.
Generación y visualización de reportes	Se implementó el módulo de historial y la exportación de reportes a PDF utilizando la librería jsPDF	Completado.
Módulo de Usuario	Se implemento un modulo de control de usuarios para su control minimo.	Parcialmente completado



Universidad Tecnológica de El Salvador

Evidencia:

Login admin

← → ↻ http://localhost:5173/login

Ingresa al Sistema Robles

Usuario:

Contraseña:

Entrar al Sistema

Transforma tu realidad。
Viví la experiencia **UTEC**



Universidad Tecnológica de El Salvador

Dashboard admin

ROBLES
PANEL ADMINISTRATIVO

Inicio

Plantillas

Auditorías

Usuarios

Reportes

acmLopez_SAS
Administrador

Cerrar Sesión

Panel de Control Operativo

Resumen de inspecciones en plantas de Robles

+ Nueva Auditoría

0

AUDITORÍAS ACTIVAS

0

AUDITORÍAS COMPLETAS

0

AUDITORÍAS ABORTADAS

Auditorías Recientes

CÓDIGO	PLANTA	FECHA	ESTADO	ACCIÓN
No hay auditorías registradas en la base de datos.				

Dashboard gestion de usuarios

Gestión de Usuarios

Administración de cuentas y accesos de Robles

+ Nuevo Usuario

3
Usuarios Totales

2
Administradores

3
Cuentas Activas

Listado de Cuentas

Username	Rol	ID Empleado	Estado	Acciones
acmLopez_SAS	Admin	72	Activo	Editar
mjfPerez_SAS	Auditor	74	Activo	Editar
rapVelazquez_SAS	Admin	12	Activo	Editar

Transforma tu realidad.
Viví la experiencia **UTEC**



Universidad Tecnológica de El Salvador

Gestion de plantillas

ROBLES
PANEL ADMINISTRATIVO

Inicio
Plantillas
Auditorías
Usuarios
Reportes

acmLopez_SAS
Administrador
Cerrar Sesión

Gestión de Plantillas

Template Management System

Buscar plantillas...

+ CREAR NUEVA PLANTILLA

CÓDIGO	NOMBRE	VERSIÓN	ACCIONES
AUD-	Lista de verificación de auditoría de procesos de fabricación	1.0	
5S-PLANTA-01	Auditoría de Metodología 5S	1.0	

Creacion de plantillas

Nueva Versión de Plantilla

Template Form Builder

GUARDAR PLANTILLA

Información de la Plantilla

TEMPLATE METADATA

CÓDIGO DE PLANTILLA: ej. AUD-001

NOMBRE DE LA PLANTILLA: ej. Seguridad Industrial

VERSIÓN: 1.0

DESCRIPCIÓN / ALCANCE DE AUDITORÍA

Opcional: Define el alcance general de esta versión...

Lista de Verificación

CHECKLIST QUESTIONS BUILDER

1 pregunta

1. Ingrese la pregunta de verificación...

Transforma tu realidad.
Viví la experiencia **UTEC**



Universidad Tecnológica de El Salvador

Historial de reportes

ROBLES
PANEL ADMINISTRATIVO

[Inicio](#)
[Plantillas](#)
[Auditorías](#)
[Usuarios](#)
[Reportes](#)

acmLopez_SAS
Administrador

Cerrar Sesión

Historial de Reportes

Consulta general de auditorías finalizadas y abortadas

Fecha Inicio
mm/dd/yyyy

Fecha Fin
mm/dd/yyyy

Estado
Todos

Buscar

Resultados

Código	Fecha	Planta	Auditado	Nota	Estado	Acciones
AUD-1779397113317-575	5/21/2026	Planta Robles	Elmer Josue Menéndez Reinosa	60%	REPROBADA	Ver Detalle
AUD-1779395275001-551	5/21/2026	Planta Robles	Elmer Josue Menéndez Reinosa	100%	APROBADA	Ver Detalle
AUD-1779395049709-553	5/21/2026	Planta Robles	Elmer Josue Menéndez Reinosa	0%	SIN_EVALUACION	Ver Detalle

detalles de un reporte

ROBLES
PANEL ADMINISTRATIVO

[Inicio](#)
[Plantillas](#)
[Auditorías](#)
[Usuarios](#)
[Reportes](#)

acmLopez_SAS
Administrador

Cerrar Sesión

Reporte de Auditoría

AUD-1779397113317-575

[← Volver](#)[Descargar PDF](#)[Enviar por Correo](#)

Datos de la Evaluación

Plantilla: Auditoría de Metodología 5S
Auditor: Marta Julia Flores Perez
Auditado: Elmer Josue Menéndez Reinosa
Fecha Fin: 5/21/2026, 3:06:34 PM

Ubicación

Planta: Planta Robles
Área: Area de Sistemas

Resultados

Estado: FINALIZADA
Calificación: 60%
Resolución: REPROBADA

Transforma tu realidad.
Viví la experiencia **UTEC**



Universidad Tecnológica de El Salvador

Manuales

Manual de Usuario:

El presente manual describe los flujos principales para la operación diaria dentro del Sistema Interno de Auditoría (SAR).

A. Cómo iniciar sesión

1. En la pantalla de autenticación, introduzca su Usuario (ej. acmLopez_SAS).
2. Ingrese su Contraseña asignada.
3. Presione el botón Iniciar Sesión. El sistema validará su rol y le dará acceso al Dashboard principal.

B. Cómo crear una nueva Plantilla de Auditoría

1. En el menú lateral izquierdo, seleccione el módulo Plantillas.
2. Dé clic en el botón Crear Plantilla o Nueva Plantilla.
3. Complete los campos de información general (Nombre de la plantilla, Versión y Código).
4. Utilice la opción Agregar Nueva Pregunta para estructurar la lista de verificación (checklist) que usarán los auditores en planta.
5. Presione Guardar Plantilla para registrarla en el sistema.



Universidad Tecnológica de El Salvador

C. Cómo consultar y descargar un Reporte en PDF

1. Diríjase al módulo de Reportes desde el menú lateral.
2. Visualice la tabla de *Historial de Reportes* y localice la auditoría que desea consultar.
3. Haga clic en el botón de Detalle de la fila seleccionada.
4. En la pantalla de información detallada, ubique y presione el botón Descargar PDF.
5. El sistema generará automáticamente el documento y lo guardará en su dispositivo.
6. También encontrara la opción de enviar reporte via Correo electrónico, presione el botón de enviar por correo y digite el correo al cual hacer llegar el reporte y clic en confirmar envió

Manual Técnico:

Este apartado detalla los requerimientos, la configuración inicial y las instrucciones de despliegue para los entornos de desarrollo o producción.

A. Requisitos del Sistema

- Entorno de ejecución: Node.js (v18.0 o superior).
- Gestor de paquetes: pnpm (incluido con Node.js).
- Base de Datos: Motor Postgre SQL en ejecución local o remota.
- Control de versiones: Git.



Universidad Tecnológica de El Salvador

B. Configuración de Variables de Entorno Tanto el *Frontend* como el *Backend* requieren configuración de entorno. Se debe crear un archivo llamado `.env` en la raíz del backend carpeta basándose en la estructura siguiente:

Variables esenciales del Backend:

Fragmento de código

PORT=3000

DB_HOST=localhost

DB_USER=root

DB_PASSWORD=su_password

DB_NAME=sistema_auditoria_robles

JWT_SECRET=clave_secreta_para_tokens

EMAIL_USER=correo_oficial_del_proyecto

EMAIL_PASS=contraseña_de_aplicacion

C. Scripts de Base de Datos Para inicializar la estructura de datos:

1. se adjuntara un script junto a la entrega debido al gran tamaño de el script sql.



Universidad Tecnológica de El Salvador

D. Instrucciones de Ejecución Para levantar el proyecto localmente, abra dos terminales y ejecute los siguientes comandos:

Para el Backend:

```
cd backend
```

```
pnpm install
```

```
pnpm run dev
```

(El servidor indicará en consola: "Servidor del SAR corriendo en http://localhost:3000")

Para el Frontend:

```
cd frontend
```

```
pnpm install
```

```
pnpm run dev
```



Universidad Tecnológica de El Salvador

Gestion de Proyecto

En esta sección explicamos cómo nos organizamos como equipo, qué hicimos en cada mes, cuáles faltan y qué aprendimos durante el proceso. El proyecto lo empezamos en febrero de 2026 y hasta el día de hoy (22/05/2026) seguimos trabajando en algunos detalles.

A. Cronograma

El cronograma muestra las fases principales del proyecto con sus fechas. Nos reuníamos una vez a la semana para ver avances e intentar repartir tareas. Siempre se nos dificultó organizarnos, pero logramos sacar adelante el proyecto.

Fase	Fecha inicio	Fecha fin	Descripción de lo que hicimos
Inicio del proyecto y primeras ideas	01/02/2026	07/02/2026	Nos reunimos para entender qué nos pedían. Leímos el enunciado varias veces y cada uno propuso ideas. Definimos que haríamos un sistema de auditorías para una empresa textil.
Levantamiento de requisitos y encuestas	08/02/2026	14/02/2026	Investigamos cómo funcionan las auditorías. Hicimos una encuesta falsa pero realista para simular que hablamos con usuarios. Sacamos los requisitos funcionales y no funcionales.
Primera entrega (documentación inicial)	15/02/2026	15/02/2026	Entregamos el primer documento con objetivos, planteamiento del problema, stakeholders y



Universidad Tecnológica de El Salvador

			requerimientos. Fue nuestra primera calificación parcial.
Diseño de pantallas (bocetos en Figma)	16/02/2026	28/02/2026	Diseñamos cómo se vería el sistema. Hicimos pantallas de login, dashboard, registro de auditorías y reportes. Usamos Figma para que todos pudieran ver los diseños.
Configuración del entorno y base de datos	01/03/2026	10/03/2026	Instalamos Node.js, React, PostgreSQL. Creamos la base de datos con tablas de usuarios, auditorías, hallazgos y plantillas. Aprendimos a conectar todo.
Programación del backend (Node.js)	11/03/2026	25/03/2026	Codificamos el servidor. Hicimos los endpoints para login, registro de usuarios, guardar auditorías y obtener hallazgos. Primeros Intentos de probar Postman.
Segunda entrega (avance de código)	20/03/2026	20/03/2026	Entregamos un avance con el backend funcionando parcialmente y los primeros prototipos. Mostramos que el login ya servía.
Programación del frontend (React)	26/03/2026	10/04/2026	Hicimos las pantallas en React. Conectamos el frontend con el backend. Ya se podía iniciar sesión, registrar auditorías y ver hallazgos.



Universidad Tecnológica de El Salvador

Tercera entrega (arquitectura y diagramas)	13/04/2026	13/04/2026	Entregamos diagramas de casos de uso, clases, secuencia y entidad-relación. También explicamos por qué usamos React, Node.js y PostgreSQL.
Pruebas internas y corrección de errores	14/04/2026	23/05/2026	Probamos el sistema entre todos. Cada uno usaba el sistema como si fuera auditor. Encontramos errores de validación, botones que no funcionaban y problemas con los permisos de roles. Los fuimos corrigiendo.
Implementación de mejoras (reportes, plantillas)	01/05/2026	23/05/2026	Terminamos la generación de reportes en PDF y la configuración de plantillas de checklist. Esto era importante porque sin reportes no sirve el sistema.
Últimos ajustes y documentación final	23/05/2026	23/05/2026	Revisamos que todo funcione. Corregimos detalles pequeños como textos mal escritos, colores feos y mensajes de error poco claros. También escribimos esta documentación final.

Comentario final del cronograma:
El proyecto ha avanzado bien. Las entregas parciales nos ayudaron a no atrasarnos. Lo que más tiempo nos llevó fue la programación del backend porque algunos no sabían bien Node.js. Lo que menos tardó fue el diseño porque usamos Figma y eso fue rápido. Hoy 22/05/2026 estamos terminando los últimos detalles de la



Universidad Tecnológica de El Salvador

documentación y puliendo algunas funcionalidades menores. Si hubiera que poner una fecha de entrega final real, sería hoy.

B. Backlog final (Hecho / Pendiente)

El backlog es nuestra lista de tareas. Aquí separamos lo que ya está terminado y funcionando de lo que todavía falta o no alcanzamos a hacer. Esto nos ayuda a ser honestos y a mostrar qué sí sirve y qué no.

Hecho (terminado)

Todo esto ya está programado, probado y funcionando. Un auditor o administrador puede usarlo sin problemas.



Universidad Tecnológica de El Salvador

Módulo	Funcionalidad	Explicación
Login	Inicio de sesión con correo y contraseña	El usuario escribe su correo y contraseña, el sistema revisa si existen y lo deja entrar. Si los datos están mal, muestra un mensaje de error.
Seguridad	Contraseñas encriptadas (hash)	Las contraseñas no se guardan como texto plano. Se guardan encriptadas usando un algoritmo llamado bcrypt. Así si alguien ve la base de datos, no puede saber las contraseñas.
Roles	Admin y Auditor tienen accesos diferentes	El administrador puede crear usuarios y ver configuraciones. El auditor solo puede hacer auditorías y ver reportes. No pueden hacer lo mismo.
Usuarios	El Admin puede crear, editar y desactivar usuarios	El administrador tiene una pantalla donde ve la lista de usuarios. Puede agregar nuevos, editar datos o desactivar a alguien para que no pueda entrar al sistema.
Empleados	Registro y consulta de empleados	Se pueden guardar los datos de los empleados que participan en las auditorías (nombre, área, puesto). Después se pueden buscar y ver.
Auditorías	Registro de auditorías por planta y área	El auditor selecciona la planta y el área que va a auditar. El sistema guarda la fecha, hora y quién la hizo. Después se puede continuar más tarde.
Hallazgos	Registro de observaciones y no conformidades	Dentro de una auditoría, el auditor puede escribir observaciones. Por ejemplo: "Falta equipo de seguridad" o "Procedimiento no cumplido". Todo se guarda en la base de datos.



**Universidad Tecnológica
de El Salvador**

Plantillas	Creación de checklists por tipo de auditoría	El administrador puede crear listas de preguntas o puntos a revisar. Por ejemplo, para una auditoría de seguridad puede tener 10 preguntas. El auditor responde sí/no o escribe observaciones. Esto ya está terminado.
Reportes	Generación de reportes en PDF	Al finalizar una auditoría, el sistema genera un reporte en PDF. Se puede descargar e imprimir.

Pendiente (En proceso en futuras actualizaciones)

Estas funcionalidades las teníamos planeadas desde el principio, pero por falta de tiempo o porque eran más difíciles de lo que pensamos, no las terminamos. Las dejamos como tareas para hacer después del curso.

Módulo	Funcionalidad	Por qué no se terminó
Notificaciones	Mandar reportes por correo automáticamente	Configurar el envío de correos con adjuntos resultó más complicado de lo que creímos. Requería configurar un servicio de correo y manejar colas de envío. No nos dio tiempo.
Reportes avanzados	Exportar a Excel o Word	Generar PDF ya lo logramos, pero pasar los mismos datos a Excel o Word requiere librerías diferentes. Lo dejamos como mejora futura.
Filtros	Buscar auditorías por fecha, planta o estado	El historial de auditorías muestra todas, pero no se pueden filtrar fácilmente. Toca agregar una barra de búsqueda con filtros. No es difícil, pero no alcanzamos.
Seguridad	Cierre de sesión por inactividad	El sistema no cierra la sesión automáticamente si te quedas mucho tiempo sin hacer nada. Esto es importante para seguridad, pero lo dejamos pendiente.
Rendimiento	Que cargue más rápido	A veces el dashboard tarda más de 3 segundos en cargar, especialmente cuando hay muchas auditorías guardadas. Hay que optimizar las consultas a la base de datos, pero no nos dio tiempo.



Universidad Tecnológica de El Salvador

Comentario final del backlog:
El sistema ya cumple con lo básico para que un auditor pueda trabajar. Las funcionalidades que están hechas son estables y se pueden usar. Las que están pendientes no impiden el uso del sistema, pero son cosas que estarían buenas agregar en el futuro para que la experiencia sea mejor.

Lecciones aprendidas

Esto es lo más valioso del proyecto: lo que aprendimos de nuestros propios errores. Aquí van 5 lecciones que nos sirvieron y que vamos a aplicar en el futuro.

1. No definimos bien los roles al principio

Qué pasó exactamente:
Cuando empezamos el proyecto, cada quien hizo lo que se le ocurrió. No hubo una reunión para decir "tú haces esto, tú haces lo otro". Algunos compañeros se pusieron a hacer la misma pantalla de login sin saber que otro ya la había empezado. También hubo tareas que nadie hizo porque todos pensaban que alguien más las iba a hacer.

Consecuencia:

Perdimos como una semana entera en trabajo duplicado. Tuvimos que sentarnos a comparar qué había hecho cada quién y decidir cuál de las dos versiones de login quedarse. También nos dimos cuenta que nadie había documentado nada porque todos pensaban que otro lo haría.

Qué haríamos diferente:
Antes de escribir una sola línea de código, hacer una lista de tareas en un Excel o Trello. Asignar cada tarea a una persona específica. Revisar cada semana que todos estén haciendo lo que les tocaba. Así nadie se mete en tareas de otros y todo el mundo sabe qué hacer.

2. Probábamos todo hasta el final

Qué pasó exactamente:
Entre abril y mayo estuvimos programando. Nadie se detenía a probar si lo que acababa de hacer funcionaba bien. Decíamos después lo pruebo o en las pruebas finales vemos. El resultado fue que acumulamos decenas de errores pequeños:

botones
hacían
campos
se



que no
nada,
que no

Universidad Tecnológica de El Salvador

guardaban, mensajes de error que salían mal y consultas que no se mostraban.

Consecuencia:

La semana de pruebas fue un caos. Estuvimos hasta tarde todos los días corrigiendo cosas que podríamos haber arreglado en el momento. Nos estresamos un montón y casi no dormimos.

Qué haríamos diferente:
Probar cada pantalla o funcionalidad justo después de terminarla. Por ejemplo, si alguien termina el login, ese mismo día prueba que funcione con varios casos: correo correcto, correo incorrecto, contraseña vacía, etc. Si encuentra un error, lo arregla ahí mismo, no lo deja para después. Así al final solo quedan errores difíciles, no un montón de tonterías.

3. Los mensajes de commit no servían para nada

Qué pasó exactamente:
En GitHub, cuando subíamos cambios, escribíamos sin estándares. Había cosas como "cambio", "arreglo", o directamente las ramas llevaban nuestros nombres. A veces ni siquiera poníamos mensaje. Esto pasaba porque teníamos prisa y no nos tomábamos un minuto para escribir algo útil.

Consecuencia:

Cuando algo dejaba de funcionar, era imposible saber qué commit había causado el error. Había que revisar archivo por archivo, línea por línea. También cuando alguien nuevo se unía al proyecto, no entendía el historial. Perdimos horas buscando errores que podrían haberse encontrado en segundos si los mensajes hubieran sido claros.

Qué haríamos diferente:
Ponernos de acuerdo en un formato simple. Por ejemplo, empezar el mensaje con un verbo: "Agrega pantalla de login", "Corrige error en registro de hallazgos", "Actualiza estilos del dashboard". Así de simple. Y obligarnos a que nadie suba un commit sin mensaje o con mensaje vago. Si alguien lo hace, el resto le dice y lo corrige.

4. Los



diseños

Universidad Tecnológica de El Salvador

cambiaron mil veces

Qué pasó exactamente:
Empezamos a programar las pantallas basándonos en dibujos rápidos que hicimos en una hoja, después nos inspiramos en figma, pero lo hicimos mas por terminar, y el tiempo nos consumio y no pudimos dar un estilo decente.

Consecuencia:
Tuvimos que rediseñar y reprogramar pantallas por funcionalidades también.

Qué haríamos diferente:
Antes de programar, hacer los diseños en Figma (que es gratuito y fácil de usar), y seguirlo de verdad organizando mejor el tiempo.

5. Aprendimos a usar ramas de GitHub (a las malas)

Qué pasó exactamente:
Al principio del proyecto, todos los integrantes trabajábamos directamente sobre la rama principal del repositorio, la que se llama "main". Cada quien hacía cambios en su computadora y los subía. El problema es que nos la pasamos mal con la organización.

Consecuencia:
Perdimos días enteros resolviendo conflictos. Había que reunirse, comparar líneas de código, decidir manualmente cuál versión servía. También hubo momentos en que no podíamos avanzar porque el sistema estaba roto y nadie sabía quién había subido el error.

Qué haríamos diferente:
Usar ramas individuales. Cada vez que alguien va a trabajar en una tarea (por ejemplo, "arreglar el login"), crea una rama nueva con un nombre como "fix-login". Trabaja ahí tranquilo sin afectar a los demás. Cuando termina, abre un Pull Request en GitHub y otro compañero revisa el código. Si está bien, se fusiona con la rama principal. Esto lo empezamos a aplicar desde la segunda entrega y funcionó muchísimo mejor. Los conflictos no desaparecieron del todo por mala organización de tiempo pero mejoramos en comparación.



Universidad Tecnológica de El Salvador

Conclusión de las lecciones aprendidas:

Este proyecto nos enseñó que la organización temprana y las buenas prácticas (probar pronto, usar ramas, mensajes de commit claros, validar diseños antes de codificar) hacen que todo sea más fácil y menos estresante. Al principio la llevamos mal, pero después aprendimos y mejoramos, aunque esto nos consumió mucho tiempo que entregar un primer prototipo 100 por ciento funcional no fue posible. Para el próximo proyecto, vamos a aplicar todo esto desde el día uno. También nos llevamos la enseñanza de que trabajar en equipo no es solo dividir tareas, sino comunicarse bien y ayudarse unos a otros.

Enlace



Universidad Tecnológica de El Salvador

Repositorio de Github: https://github.com/EJMR18/Sistema_Auditoria_Robles

Transforma tu realidad.
Viví la experiencia **UTEC**